

Course: Research Methodology

An LLM-Based Bilingual Campus Assistant for Academic Service Access via RAG and Quiizy Integration: Design, Implementation, Cloud Deployment and Evaluation in a Central African University

Moutcheu Somo Gift Anderson
Abena Alex Nelson Ryan
Foning Lackmata Luc-Xavier
Fotsing Kengne Weber
Chengho Fongang Marie Louise
Kouetche Simo Yann
Takam Sokoudjo Ismael Ulrich
Student ID: Your ID

Supervisor:
Dr.-Ing. Philippe Tamla
Professor for Data Science and Artificial Intelligence
PostDoc Researcher at
Department of Multimedia and Internet Applications
Faculty of Mathematics und Computer Science
Universitätsstr. 1
58097 Hagen

Abstract

This paper describes the design, implementation, and evaluation of **AcadAi**, a web-based Smart Campus system developed for university students. The system combines a **Large Language Model (LLM)** with a **Retrieval-Augmented Generation (RAG)** pipeline to offer bilingual academic and administrative service access in a Central African university setting. Integration with **Quiizy**, the institution's academic platform for student assessments and course information, grounds the system in real institutional data. Evaluation employed a quantitative student satisfaction survey based on the **Technology Acceptance Model (TAM)**.

Contents

1 Introduction **1**

2 State of the Art in Science and Technology **2**

2.1 AI Chatbots and Conversational Agents in Higher Education . . . 3

2.2 Large Language Models for Natural Language Understanding . . . 3

2.3 Retrieval-Augmented Generation for Institutional Knowledge Access . . . 4

2.4 Integration of AI Chatbots with Learning and Assessment Platforms . . . 4

2.5 Comparable University AI Assistant Systems . . . 5

2.6 Evaluation Frameworks for AI-Based Educational Systems . . . 5

2.7 Summary and Research Gap . . . 5

3 Design and Modeling **6**

3.1 The Existing System and the Design Context . . . 6

3.2 Use Case Model . . . 6

3.3 System Architecture . . . 7

3.4 MVC Architecture and Processing Pipeline . . . 7

3.5 Information Model . . . 9

3.6 Technology Choices and Design Rationale . . . 9

4 Implementation **11**

4.1 Backend Services and API Layer . . . 11

4.2 Mobile Application: Offline-First Design . . . 12

4.3 Web Application: Teacher and Academic Administrator Dashboard . . . 14

4.4 Retrieval-Augmented Generation Pipeline . . . 15

4.5 Chat Interface Design and RAG Integration . . . 16

4.6 Cloud Deployment and Infrastructure . . . 16

4.7 Bilingual Support: Chat Interface . . . 17

4.8 Addressing Remaining Challenges Through Implementation . . . 17

4.9 Summary . . . 17

5 Evaluation **17**

5.1 Evaluation Methodology . . . 17

5.2 Survey Design and Administration . . . 18

5.3 Participant Characteristics . . . 18

5.4 Quantitative Results . . . 18

5.4.1 Descriptive Statistics . . . 18

5.4.2 TAM Dimension Analysis . . . 18

5.4.3 Response Distribution . . . 18

5.4.4 Demographic Cross-tabulation . . . 19

5.5 Discussion of Findings . . . 19

5.5.1 Ease of Use and Learnability . . . 19

5.5.2 Perceived Usefulness and Speed of Access . . . 19

5.5.3 Bilingual Effectiveness . . . 19

5.5.4 Accuracy and Content Freshness . . . 20

5.5.5 Findings Summary and Research Question Responses . . . 20

6 Conclusion and Future Work **20**
6.1 Research Contributions 20
6.2 Evaluation Insights 20
6.3 System Limitations 21
6.4 Future Work Directions 21
6.5 Final Remarks 21

References **22**

List of Figures

1 Use case diagram of AcadAi 7
2 Web-based conversational interface for student academic service access 8
3 Component architecture of the extended Quiizy campus assistant 8
4 Extended MVC architecture showing the AI layer 9
5 Extended information model showing new entities in relational context 10
6 Quiizy login interface: teacher and administrator authentication with role selection 12
7 Quiizy administration dashboard: semesters management interface for academic year configuration 13
8 Quiizy administration dashboard: teaching units management for course organization . . . 13

List of Tables

1 Likert-scale responses: Mean, standard deviation, range, and sample size (n=23) 19

List of Listings

1 Sync queue processing: core operation from syncQueueOperations.ts 14
2 Course material upload endpoint: Angular service integration 15
3 RAG retrieval endpoint: core function from FastAPI service 16

1 Introduction

Universities in developing countries increasingly rely on web-based digital services to support students. Typical services include retrieving academic information, for example course details and announcements, completing administrative tasks, finding schedules, and getting study support. As enrolment grows and support offices remain limited, students often face long response times and fragmented information sources. Across Africa, these difficulties are intensified by infrastructure constraints and uneven access conditions. In Cameroon, they are further amplified by a bilingual academic environment in which students may need the same service in both English and French. To make such services available at scale, universities increasingly host applications on cloud computing platforms such as Amazon Web Services (AWS). In this paper, cloud computing means delivering computing resources such as servers, storage, and databases over the internet on demand, rather than running and maintaining all infrastructure locally [1, 2].

This work focuses on a web-based campus assistant for student services. In this paper, the assistant is powered by **Artificial Intelligence (AI)**, meaning computer methods that can perform tasks that normally require human intelligence such as language understanding. We therefore refer to the system as **AcadAi**: a web-based student support system that answers questions and helps students reach campus services through natural-language dialogue. A key component is a **Large Language Model (LLM)**, a deep-learning model trained on large text collections that can generate fluent text responses [3, 4]. When an LLM is placed inside a dialogue interface it becomes a conversational agent, or chatbot, which interacts with users through conversation instead of menus [5, 6]. Effective campus assistants often need more than question-answering. This paper therefore also uses a **Retrieval-Augmented Generation (RAG)** pipeline, which combines generation with retrieval by first searching a verified document collection and then generating an answer using the retrieved passages as evidence [7, 8]. The assistant is also designed to connect to **Quiizy**, the academic platform in the target institution through which students take multiple-choice tests and access related course information. This explicit platform scope is retained to satisfy the reviewer comment asking which specific assistant and which concrete LMS-like integration are being addressed.

MS1: Smart campus systems can improve student engagement when they make academic information easier to access and use. Student engagement is understood here as students' effective interaction with academic resources, deadlines, and support services needed for their studies. Evidence suggests that smart campus systems embedding AI services can improve engagement by reducing information friction and making institutional resources more responsive [9]. **MS2: AI-based campus assistants require rigorous usability and acceptance evaluation before they can be considered genuinely helpful.** Response delay in this paper refers to the time students experience between requesting information and obtaining a useful answer, whether from human staff or digital systems. Reducing this delay matters only if the system is also understandable, learnable, and trusted, which is why usability and acceptance evaluation remain central [10, 11, 12].

The overall goal of this research is to design, implement, and evaluate a web-based system, **AcadAi**, that provides bilingual academic and administrative support using an LLM-based conversational interface, a RAG-based retrieval component, and integration with the Quiizy platform context. **PS1:** The existing literature documents few bilingual, rigorously-evaluated campus assistants for Central African universities that ground answers in institutional sources via LLM and retrieval [5, 6, 7, 8]. **PS2:** When prototypes do exist elsewhere, evaluation often relies on informal feedback rather than structured usability inspection and acceptance frameworks. This gap makes it hard to determine whether these systems are genuinely easy to learn and actually useful [10, 11, 12, 13, 14]. Both factors point to the need for a context-specific, rigorously-evaluated system.

These problems motivate two core research questions: **RQ1:** How can you architect, build, and deploy **AcadAi**, a web-based system that integrates an LLM, RAG-enabled retrieval, and Quiizy academic content on a cloud platform, in order to serve bilingual students? **RQ2:** Does the resulting system actually meet usability standards, and what is its impact on student satisfaction and information

access?

This research follows the well-established Nunamaker systems development research framework [15], a structured approach for developing information systems consisting of four phases: **Observation**, **Theory Building**, **Implementation**, and **Experimentation**. Following this methodology, the research objectives are grouped by research question and expressed in the Nunamaker categories of Observation (**O**), Theory Building (**TB**), Implementation (**I**), and Experiment (**E**). The objectives related to **RQ1** are as follows: **RO1.1–O:** Review existing methods and tools for AI chatbots, conversational agents, LLMs, RAG architectures, cloud deployment, and platform integration in higher education, especially in developing and multilingual contexts. **RO1.2–TB:** Derive the design requirements and system model for **AcadAi** based on an LLM, a RAG-enabled retrieval pipeline, cloud deployment support, and integration with Quiizy. **RO1.3–I:** Implement a bilingual web-based prototype that supports English and French interaction for academic service access and deploy on AWS cloud infrastructure. **RO1.4–E:** Evaluate whether the implemented prototype improves access to academic information and task support for representative student use cases. The objectives related to **RQ2** are as follows: **RO2.1–O:** Review usability issues, interaction challenges, and student expectations relevant to AI-based educational assistants. **RO2.2–TB:** Define an evaluation model grounded in usability, technology acceptance, and information systems success theory. **RO2.3–I:** Instrument the prototype and evaluation materials so that usability and student-satisfaction data can be collected systematically. **RO2.4–E:** Conduct a student satisfaction study to assess learnability, perceived usefulness, perceived ease of use, and overall service quality.

To answer the research questions, the study proceeds in a simple and traceable way. Observation activities identify the student information-access constraints and bilingual service needs. Theory building translates these into system and evaluation requirements. Implementation realises a bilingual web prototype, and experimentation assesses whether the resulting artefact is usable and useful enough to address the original problems. The remainder of this paper is organised as follows. Chapter 2 reviews related work on conversational agents in higher education, LLMs, RAG, platform integration, and evaluation frameworks. Chapter 3 presents the system design and modelling. Chapter 4 describes the implementation. Chapter 5 reports evaluation results. Chapter 6 concludes with limitations and future work. Chapter 2 is structured to support the design choices made later in the paper: it surveys chatbot use in universities, reviews LLM foundations and limitations, introduces RAG for grounded institutional question answering, analyses LMS and platform integration patterns, compares related university assistant systems, and summarises evaluation frameworks combining usability inspection and acceptance models.

2 State of the Art in Science and Technology

This chapter presents the state of the art in scientific and technological approaches directly relevant to the design and evaluation of a web-based, bilingual, LLM-based campus assistant integrated with an academic platform for university students in a Central African context. The review supports the observation phase of the Nunamaker methodology [15] and specifically addresses research objectives RO1.1–O and RO2.1–O by documenting student information-access needs, surveying relevant technological solutions, and identifying their limitations.

The chapter is organised as follows. Section 2.1 surveys AI chatbots and conversational agents in higher education. Section 2.2 examines LLMs as the enabling technology for natural language understanding and generation. Section 2.3 presents RAG as the principal architectural approach for grounding responses in verified institutional knowledge. Section 2.4 analyses chatbot integration with learning and assessment platforms, including the type of functionality represented here by Quiizy. Section 2.5 reviews comparable university AI assistant systems. Section 2.6 discusses evaluation frameworks suitable for AI-based educational systems. Based on this review, Section 2.7 formulates the remaining challenges that motivate the modeling in Chapter 3.

2.1 AI Chatbots and Conversational Agents in Higher Education

This section addresses RO1.1–O by reviewing AI-powered conversational agents in higher education and establishing the documented service needs that motivate the proposed system. A conversational agent, or chatbot, is a software system designed to simulate human-like dialogue through a natural-language interface, allowing users to query information, request services, or receive guidance without navigating complex administrative workflows [5, 6]. These systems range from rule-based assistants that match input to predefined responses to more advanced LLM-powered agents capable of multi-turn dialogue, contextual interaction, and broader language understanding [16, 17].

Labadze et al. [5] review empirical evidence and find that AI chatbots can meaningfully improve student access to academic information through immediate, personalized support. Similar gains in accessibility and satisfaction appear in Pereira-Molina and colleagues' work [6], though they note that most documented systems operate in North American and European institutions. Wang et al. [16] demonstrate that conversational interfaces reduce administrative friction and enhance information access, particularly when staff capacity is limited.

Student perception research adds nuance. Stöhr and colleagues [17] identify strong dependencies between continued usage and both response quality and linguistic appropriateness. Nguyen et al. [18] push beyond simple information retrieval, arguing that conversational systems should support goal-directed tasks and multi-step dialogue—critical for students seeking Quiizy-related information, who often need both factual answers and procedural clarification.

Yet the landscape remains constrained. Most documented campus assistants operate monolingually in narrow domains within relatively stable connectivity environments. Multilingual, rigorously-evaluated assistants tailored to Central African universities are conspicuously scarce. This is significant where English and French coexist institutionally and network conditions fluctuate. We identify this gap as our first remaining challenge:

RC1: Bilingual, context-aware AI campus assistants designed and evaluated for Central African university settings remain largely undocumented.

2.2 Large Language Models for Natural Language Understanding

This section supports RO1.2–TB by reviewing LLMs as the core enabling technology for natural language understanding and generation in the proposed assistant. An LLM is a neural language model with very large numbers of trainable parameters, pre-trained on massive text corpora to acquire broad linguistic competence and general reasoning ability [3, 4]. These models are particularly well suited to dialogue systems because they can interpret open-ended user questions and produce coherent natural-language answers.

The architectural foundation of most contemporary LLMs is the Transformer, introduced by Vaswani et al. [19]. Its self-attention mechanism allows efficient modeling of contextual dependencies across a sequence, which is essential for understanding complex, multi-clause student questions. Brown et al. [4] demonstrated that sufficiently scaled language models exhibit strong few-shot behavior, which is valuable in academic service environments where it is impossible to enumerate every possible student query in advance.

Related model families further support the ecosystem around LLM-enabled assistants. Devlin et al. [20] contributed dense language representations through BERT, which remain influential in semantic retrieval and embedding pipelines, while Touvron et al. [21] expanded access to high-quality open foundation models through LLaMA. Together, these developments make it technically feasible to build assistants that combine generation, retrieval, and context management.

However, LLMs also have serious limitations in institutional service settings. First, they may hallucinate by producing fluent but incorrect or fabricated statements [3, 8]. Second, their knowledge is static: they do not automatically reflect changes in course content, deadlines, announcements, or platform usage rules unless connected to external information sources [7, 22]. These limitations are critical for an assistant that must provide dependable academic information. This leads to the second remaining challenge:

RC2: LLMs alone are insufficient for reliable in-

stitutional information access because hallucination and static knowledge must be addressed through external grounding mechanisms.

2.3 Retrieval-Augmented Generation for Institutional Knowledge Access

This section continues RO1.2–TB by presenting RAG as the main architectural approach for addressing the limitations identified above. Lewis et al. [7] introduced RAG as a framework that combines parametric model knowledge in an LLM with non-parametric knowledge retrieved from an external document collection at inference time. In a typical RAG pipeline, a user query is transformed into a vector representation, matched against an indexed document store, and the most relevant passages are then supplied to the generator as context for answer production.

For campus assistants, this architecture offers clear advantages: answers can be anchored in current institutional materials rather than relying solely on the model's internal knowledge. The RAG landscape itself is diverse. Gao et al. [8] distinguish between basic retrieval-plus-generation and more advanced modular systems supporting query rewriting, reranking, and multi-step reasoning. In educational contexts specifically, Essel et al. [23] document improvements in factual accuracy, response relevance, and satisfaction over pure LLM approaches. Soliman and colleagues [22] extend this argument, emphasizing that RAG directly mitigates hallucination, knowledge staleness, and domain-specificity challenges that plague LLM-only systems.

In the context of this paper, RAG is the mechanism through which the assistant can use Quiizy-related and institutional content to answer questions about quizzes, schedules, announcements, and related academic procedures. The approach supports the reviewer's request to clarify that the proposed intelligent component is not just an abstract autonomous agent, but a concrete RAG-based retrieval and answer-generation pipeline. At the same time, deployments in multilingual and resource-constrained African educational contexts remain limited. This supports the third remaining challenge:

RC3: RAG is well suited to institutional knowledge access, but published evidence on bilingual RAG deployments adapted to Central African university conditions remains limited.

2.4 Integration of AI Chatbots with Learning and Assessment Platforms

This section addresses RO1.1–O and RO1.2–TB by reviewing the integration of AI chatbots with learning and assessment platforms such as LMS environments and the Quiizy platform targeted in this study. Platforms of this kind manage course materials, assignments, assessment activities, grades, announcements, and related student-facing services. When a conversational assistant is connected to such a platform, it can support personalized, real-time queries that depend on institutional data rather than static general knowledge [24, 25, 26].

Al-Saqqa et al. [24] show that chatbot integration varies in both technical depth and personalization depth. Many existing systems remain at the level of document ingestion, without exploiting the richer student-specific context available through platform data. Neumann et al. [25] demonstrate that an LLM-driven chatbot integrated with an educational information environment can improve access to relevant course content through retrieval and contextual dialogue. Kim et al. [26] further show that memory-augmented language-model architectures can preserve conversational context across multiple turns, which is important when students ask follow-up questions about ongoing tasks or deadlines.

For this study, the relevant integration context is Quiizy, a platform used for student multiple-choice tests and related academic information. The proposed assistant therefore targets a concrete, bounded service domain rather than a generic or unspecified LMS. This directly answers the PDF comments requesting that the paper name the platform and the specific services under discussion. Even so, published reports of RAG-enabled assistants designed and evaluated for platform ecosystems of this kind in Francophone or bilingual Central African universities remain rare. This motivates the fourth remaining challenge:

RC4: Platform-integrated educational assistants are promising, but comparable bilingual solutions designed for Quiizy-like academic environments in Central African universities remain underexplored.

2.5 Comparable University AI Assistant Systems

This section supports RO1.1–O and anticipates RO1.4–E by reviewing comparable university AI assistant systems. Several prior systems establish technical feasibility, though each leaves open areas central to our work.

Academic advising through LLMs receives attention from Akiba and Fraboni [27], who argue for systems that support genuine student empowerment. Document-retrieval approaches appear in Nguyen et al.'s UniBot [28], which couples information retrieval with LLM generation and achieves strong relevance scores. Neupane and colleagues [29] show that aggregating diverse institutional resources into a retrieval pipeline reduces hallucination and strengthens answer quality.

Multilingual contexts are less well documented. Bilquise et al. [30] describe a bilingual academic-advising chatbot that broadens reach and usefulness for non-English speakers. This work offers a valuable precedent for the English-French requirement in Cameroon. More recently, Tamascelli et al. [31] find that agent-augmented advising can outperform pure retrieval systems on complex student queries.

These examples confirm LLM-based university assistants are viable and valuable. Yet none appear to combine the constellation of requirements addressed here: bilingual (English-French) dialogue, RAG-grounded answers, integration with a specific academic platform like Quiizy, cloud deployment, and rigorous combined evaluation within a Central African context. This gap motivates the fifth remaining challenge:

RC5: Comparable university AI assistant systems demonstrate feasibility, but not the full combination of bilingualism, RAG grounding, platform integration, and Central African evaluation required here.

2.6 Evaluation Frameworks for AI-Based Educational Systems

This section addresses RO2.1–O and supports RO2.2–TB by reviewing the evaluation frameworks employed in this research. Evaluating an AI campus assistant requires capturing both technical behavior and user-centered experience. Relevant dimensions include response quality, retrieval faithfulness, learnability, perceived usefulness, ease of use, and overall student satisfaction.

The **Cognitive Walkthrough (CW)** is a scenario-based usability inspection method in which evaluators step through tasks to determine whether a system is understandable and learnable without prior training. Hollingsed and Novick [10] show that CW reliably reveals learnability problems in goal-oriented interactive systems. Mahatody et al. [11] further classify the evolution of the method and confirm its continued relevance as a cost-effective inspection technique. Because chatbot systems rely heavily on user formulation of requests and system guidance, CW is well suited to identifying interaction breakdowns before large-scale user testing.

For user acceptance, the **Technology Acceptance Model (TAM)** proposed by Davis [12] remains foundational. TAM centers on perceived usefulness and perceived ease of use as key determinants of technology adoption. Venkatesh et al. [13] extend this line of thinking through UTAUT, adding factors such as social influence and facilitating conditions. DeLone and McLean [14] contribute a complementary information-systems success perspective through information quality, system quality, and service quality.

Used together, these frameworks provide a strong basis for evaluating whether the proposed assistant is not only technically functional but also understandable, useful, and acceptable to students. Yet combined evaluation protocols of this kind remain uncommon for bilingual, RAG-enabled, platform-integrated campus assistants in Central African contexts. This supports the sixth remaining challenge:

RC6: A rigorous combined evaluation framework is available, but it has rarely been applied to bilingual, RAG-based campus assistants in the target context.

2.7 Summary and Research Gap

This chapter has surveyed the core scientific and technological work relevant to the proposed **AcadAi** system. The evidence is clear: AI chatbots can genuinely improve academic information access, though bilingual implementations for Central Africa remain scarce (RC1). LLMs possess strong language capabilities, but they require additional mechanisms to mitigate hallucination and handle knowledge stal-

eness (RC2). RAG offers a sound architectural approach for grounding responses in institutional sources, yet bilingual, context-adapted deployments in this region are uncommon (RC3). Integration with assessment platforms like Quiizy is valuable, but rigorously evaluated solutions within this specific setting remain limited (RC4). Comparable systems elsewhere demonstrate technical feasibility, though none appear to combine the full scope of requirements (bilingualism, RAG grounding, Quiizy integration, and rigorous African evaluation) addressed here (RC5). Evaluation frameworks exist that blend usability and acceptance models, but their application to bilingual RAG-enabled assistants in this context is rare (RC6).

These six gaps collectively define the research opportunity: designing, building, deploying, and evaluating a bilingual, RAG-enabled, Quiizy-integrated system (**AcadAi**) suited to a Central African university. The following chapter translates these motivations into system design choices.

3 Design and Modeling

This chapter presents the proposed system design (Theory Building in the Nunamaker methodology [15]). The design is guided by **User-Centered Design (UCD)**: the architecture is derived from the users' tasks and constraints (mobile-first access, bilingual interaction, and limited human support capacity), and only then mapped to concrete technology components.

The proposed artefact extends the existing Quiizy platform with a bilingual (English/French) conversational assistant for academic service access. Answers are grounded in institutional course documents through **RAG** (Retrieval-Augmented Generation): the system retrieves relevant passages from a trusted document collection and uses these passages as context when generating the response.

To remain concise, technical terms are used sparingly and in a pragmatic way. A **webhook** is an HTTP endpoint that receives events (here: WhatsApp messages). An **embedding** is a numeric representation of text meaning, and a **vector store** is a database that stores embeddings to support similarity search.

3.1 The Existing System and the Design Context

The platform being extended is Quiizy, a web-based academic assessment system implemented with NestJS (Node.js) and backed by PostgreSQL [32, 33]. The data-access layer uses TypeORM [34]. The existing authentication mechanism relies on JWT (a signed token used to represent an authenticated session).

Rather than replacing Quiizy, the design adds three capabilities that map directly to user needs: (1) a teacher-facing workflow to upload course materials within the LMS, (2) a document indexing pipeline that makes uploaded materials searchable by meaning (RAG), and (3) a student-facing conversational interface that returns responses in English or French.

Note that Quiizy includes a native mobile application for students to take quizzes; this mobile app is already implemented and is outside the scope of the present research. The research focuses exclusively on the web-based conversational assistant interface for academic service access.

In UCD terms, the primary requirement is to minimise user effort while maximising access speed and clarity: students interact through a conversational interface, teachers use a small extension of the existing web workflow, and the retrieval infrastructure remains lightweight enough to deploy within constrained institutional environments.

3.2 Use Case Model

Figure 1 shows the use case model with four actors: Student, Teacher, Administrator, and an **IR Agent**. Here, **IR Agent** denotes the retrieval component responsible for selecting relevant passages from the indexed course documents.

The student's primary use case is to ask a natural-language question through the conversational interface and receive a grounded answer in the language of the request. Internally, the system executes a short pipeline: (a) retrieve relevant document passages, (b) assemble a prompt that includes the retrieved context, and (c) generate the final response using the LLM.

The teacher's primary use case is to upload course materials through the web interface. The system then extracts text and indexes it for later retrieval.

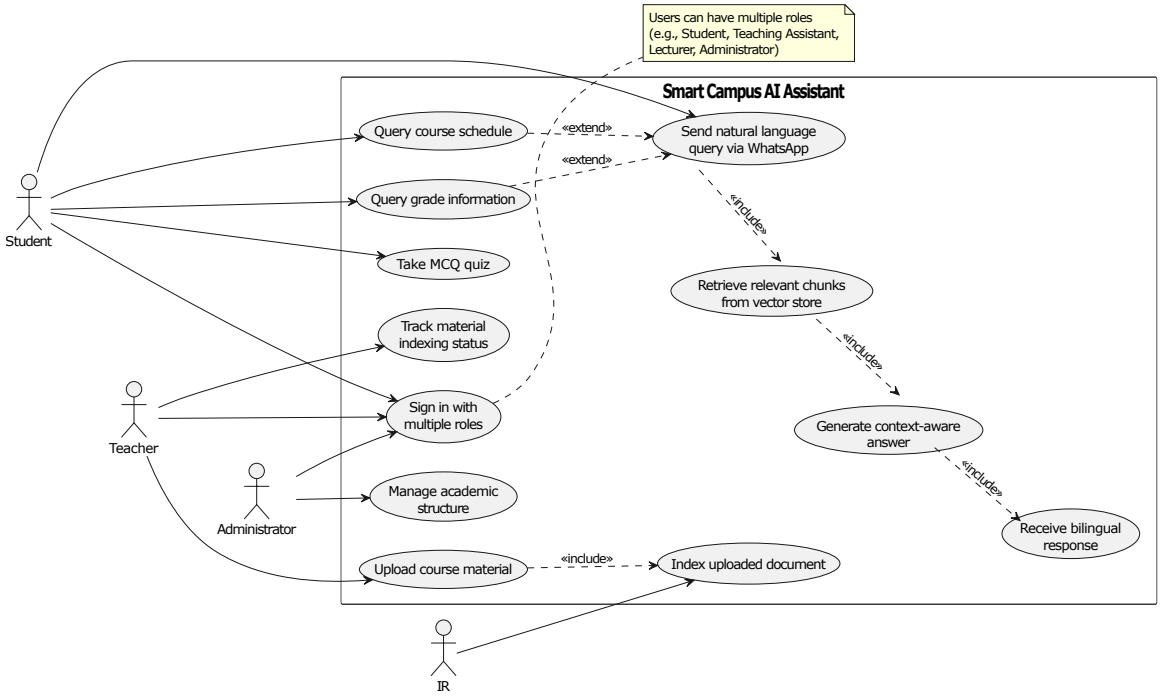


Figure 1: Use case diagram of AcadAi

The administrator continues to manage the academic structure in Quiizy without changes; the assistant features are additive.

3.3 System Architecture

The overall architecture is shown in Figure 3. The existing Quiizy backend stays in place, and new components are added around it for (1) document upload and indexing and (2) conversational chat services.

The NestJS backend coordinates the overall workflow. Three new functional areas are introduced: **Materials** (upload, persistence, and processing coordination), **Chatbot Gateway** (conversational interface webhook handling and session management), and an **LLM Orchestrator** (prompt assembly and invocation of retrieval and generation).

A separate Python FastAPI service performs the document indexing and retrieval steps [35]. This separation isolates language-processing concerns from the core LMS backend and leverages Python libraries commonly used for text extraction and semantic search.

For semantic retrieval, the system uses ChromaDB as the vector store [36, 8]. Storage is therefore split across: PostgreSQL (structured LMS data and metadata), file storage (raw uploaded documents), and ChromaDB (embeddings used for similarity search).

3.4 MVC Architecture and Processing Pipeline

Quiizy follows the standard NestJS structure: controllers receive HTTP requests, services implement logic, and repositories access the database. In this project, the AI-related work (document indexing and retrieval) is handled by the Python service. Figure 4 summarizes this split.

At the system boundary, two controllers handle external input: `ChatbotController` (conversational chat interface) and `MaterialsController` (file upload). Controllers perform request validation and delegate to services for business logic.

For chat, the service manages a conversation session, associates the phone number with a student account when possible, and queries the Python retrieval endpoint for relevant passages. The orchestrator then builds a prompt that includes conversation context, retrieved passages, and a language directive, and finally requests the LLM to generate the reply.

For documents, the Python service processes uploads by extracting text, splitting it into chunks, computing embeddings, and storing them in Chro-

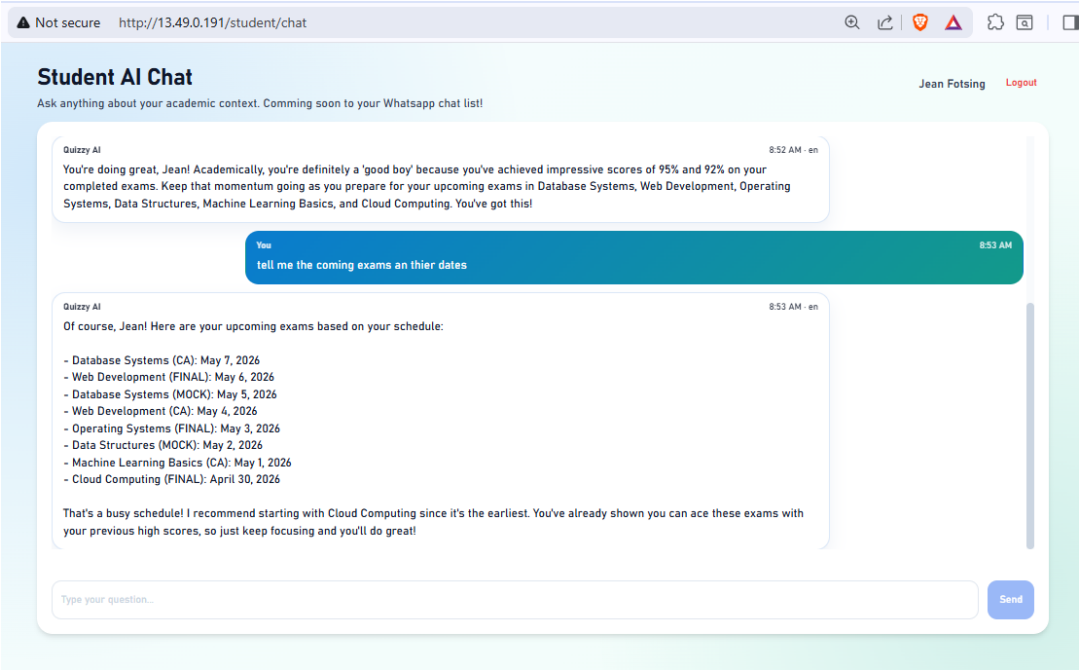


Figure 2: Web-based conversational interface for student academic service access

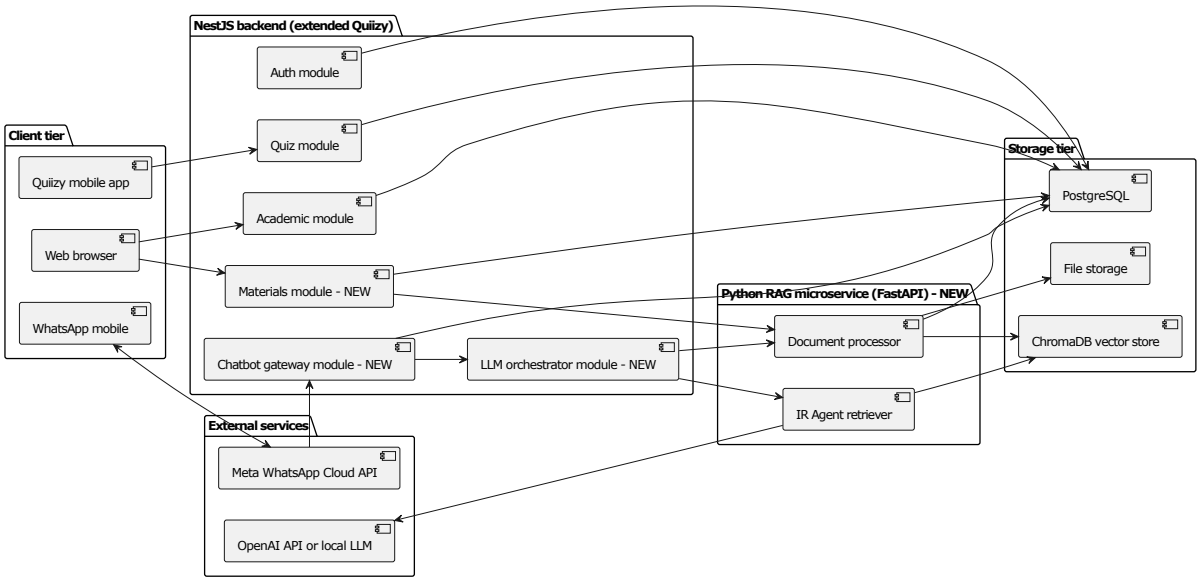


Figure 3: Component architecture of the extended Quiizy campus assistant

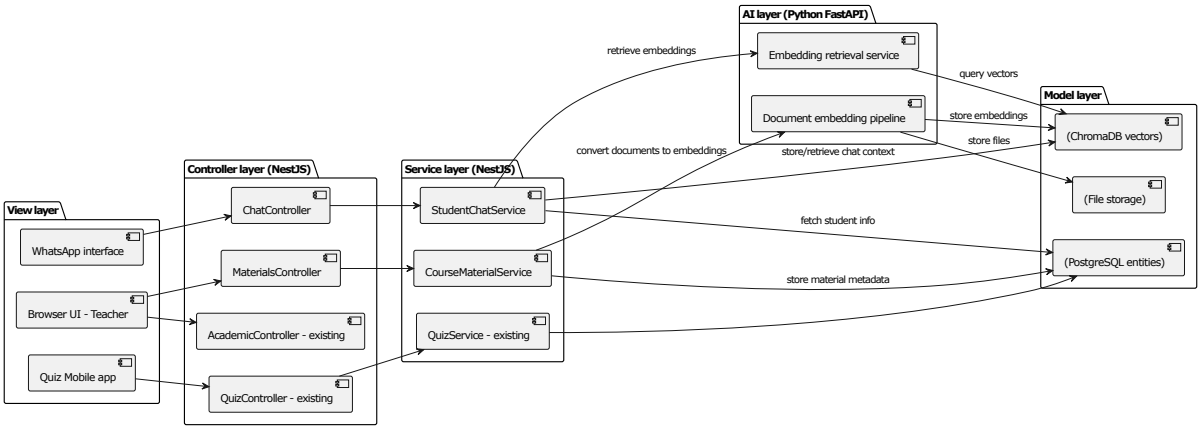


Figure 4: Extended MVC architecture showing the AI layer

maDB. For each query, it embeds the question and retrieves the most similar chunks as evidence. If retrieval does not return sufficiently relevant passages, the assistant returns a controlled fallback response to avoid unsupported statements.

3.5 Information Model

The information model, shown in Figure 5, extends the existing Quiizy data schema with two new entity groups. The existing entities for academic structure (`Course`, `Semester`, `ClassAcademicYear`, `Student`, `Teacher`) are shown to provide relational context. The new entities are `CourseMaterial`, which represents an uploaded document, and the pair `ChatSession` and `ChatMessage`, which together represent a conversational interaction.

The central new entity is `CourseMaterial`, linking an uploaded file to a course. The `processingStatus` field tracks the indexing lifecycle (`PENDING`, `PROCESSING`, `INDEXED`, `FAILED`), and `embeddingCount` records how many chunks were successfully indexed.

Chat interactions are represented by `ChatSession` and `ChatMessage`. Sessions are keyed by a unique session identifier linked to the student, and can be linked to a `Student` record to enable personalization and session history retrieval. Messages store user queries and assistant replies, and can record which materials contributed to the answer to support traceability.

ChromaDB remains external to the relational ERD: it stores embeddings and associated identifiers (e.g., `material_id`, `course_id`) so retrieved text can be traced back to the originating course document.

3.6 Technology Choices and Design Rationale

The technology choices follow the UCD requirements and the architectural goal of grounding answers in institutional sources.

Student conversational interface. The design specifies a conversational interface for students to interact with the assistant via natural dialogue, reducing onboarding effort compared to menu-driven systems. The original architectural design considered WhatsApp as a student interface to leverage mobile-first usage patterns and the Meta WhatsApp Cloud API with webhooks for receiving messages and delivering responses [37]. However, due to time constraints during implementation, the team focused on developing a web-based conversational interface that achieves the same pedagogical and functional goals while enabling rapid iteration. The web interface supports the same conversational agent architecture and multilingual response generation, preserving the core design goals without requiring mobile channel integration at this stage. WhatsApp integration remains a viable extension for future deployment phases.

Quiizy backend stack (NestJS, PostgreSQL, TypeORM). The existing platform stack is retained to minimise disruption to institutional workflows. NestJS provides a modular structure for controllers and services [32]. PostgreSQL is used as the relational data store for structured academic entities [33], and TypeORM provides an object-relational mapping layer with support for PostgreSQL and schema evolution [34].

Python retrieval service (FastAPI). The

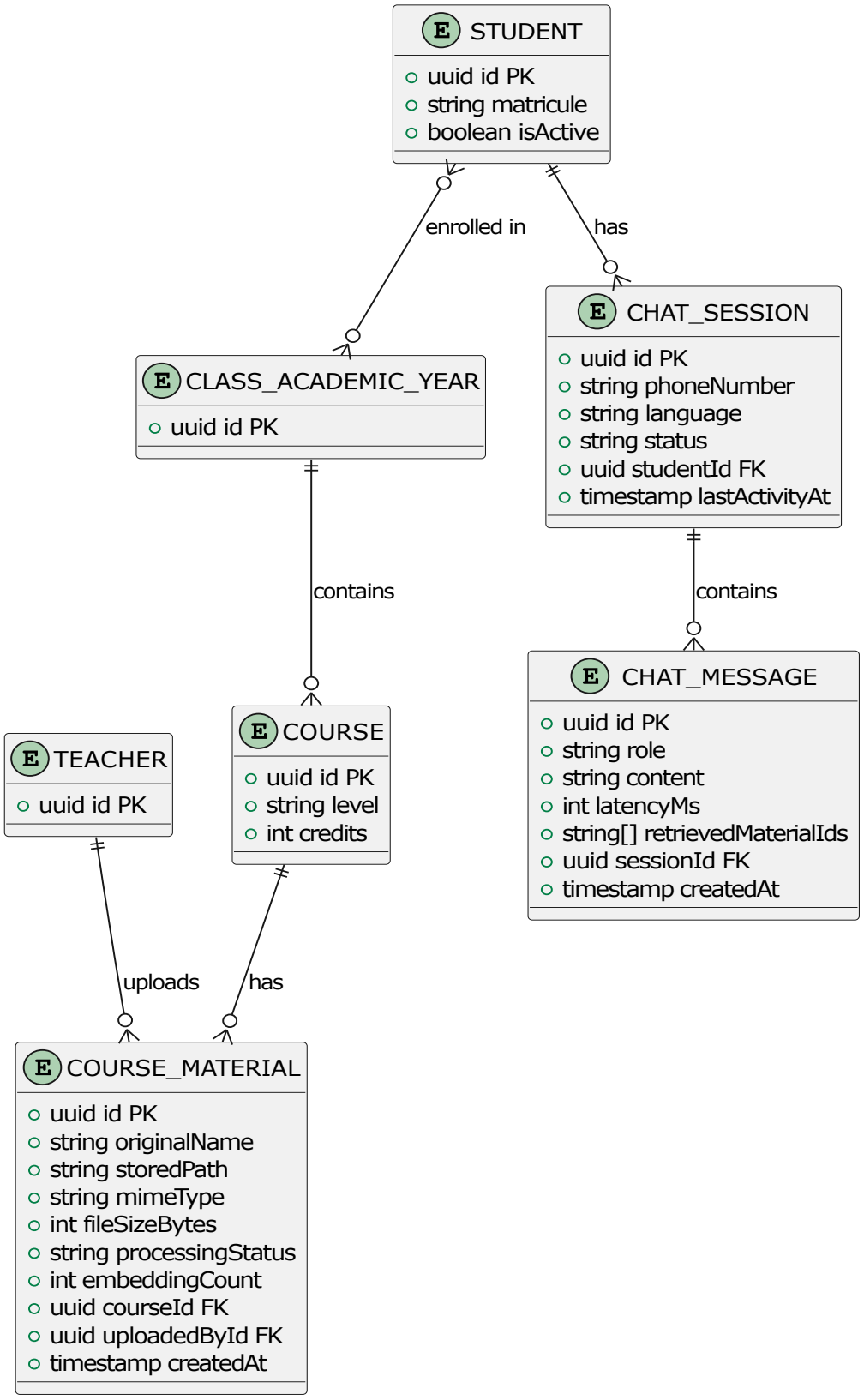


Figure 5: Extended information model showing new entities in relational context

RAG workflow combines retrieval and generation [7, 8] and benefits from Python tooling for document processing and embedding-based search. A separate FastAPI service is therefore used to encapsulate the indexing and retrieval pipeline behind a stable HTTP interface [35]. This service boundary aligns with common distributed deployment practices in cloud-based systems such as AWS, where microservices architectures enable scalable and maintainable solutions [1, 2].

Vector store (ChromaDB). ChromaDB is used to store embeddings and metadata and to support similarity-based retrieval [36]. It can be run locally or self-hosted, enabling a lightweight deployment path, and it matches the vector-store role described in RAG surveys [8].

Bilingual interaction. The assistant responds in the language of the incoming message (English or French) using a single prompt and a single LLM endpoint. This avoids maintaining parallel pipelines while still supporting bilingual access.

The next chapter describes the implementation.

4 Implementation

This chapter describes the implementation of the proposed **AcadAi** system, translating the design and modeling presented in Chapter 3 into a working prototype. The implementation is guided by the Nunamaker methodology's System Development phase and specifically addresses research objectives RO1.3–I and RO2.3–I, which call for implementing a bilingual web-based prototype with English and French support for academic service access, deploying it on AWS cloud infrastructure, and instrumenting it for systematic evaluation.

The implementation follows a pragmatic layered architecture that separates concerns across frontend (mobile and web), backend API, and specialized AI services. A key architectural principle is offline-first design: the mobile application maintains a local SQLite database for quiz data and student answers, synchronizes opportunistically with the backend via a sync queue, and remains functional even when network connectivity is intermittent. This approach is essential for the Central African university context where connectivity may be unreliable. The backend services, implemented in NestJS with PostgreSQL, maintain academic structure, authentication, and quiz management. A separate Python FastAPI microservice handles the RAG pipeline, document indexing, and LLM orchestration. Together, these components realize the design requirements while addressing the remaining challenges identified in Chapter 2, particularly RC1 (bilingual context-aware assistants for Central Africa), RC2 (avoiding LLM hallucination through retrieval grounding), and RC4 (platform integration with Quiizy).

4.1 Backend Services and API Layer

The backend is built on NestJS, a Node.js framework that provides a modular, scalable structure aligned with the existing Quiizy platform. The API layer exposes three main functional areas: academic information endpoints, authentication and session management, and quiz and assessment endpoints. All endpoints follow REST conventions and are protected by JWT-based authentication, consistent with the design in Chapter 3.

The academic information service manages the hierarchical structure of the institution: academic years, semesters, classes, courses, and teaching units. These entities map to the information model defined in Section 3.5 and are stored in a PostgreSQL relational database. A request to retrieve the current academic year for a logged-in student, for example, queries the database for the active academic year and returns it as JSON. The service also manages student-class linkages, enabling personalized course and quiz lists. The API is designed to be stateless: each request includes the student's JWT token, which the backend validates and uses to determine the requesting user's role and permissions.

Authentication is implemented via JWT tokens following industry standards. When a student logs in with email and password, the backend validates credentials against a secure password hash, generates a JWT access token with a short expiration time (typically 15 minutes), and returns a refresh token with a longer expiration (typically 7 days). The refresh token is stored securely on the client, allowing the mobile or web application to obtain a new access token when the old one expires, without requiring the user to re-enter credentials. This pattern balances security and user experience.

The quiz service manages the complete lifecycle

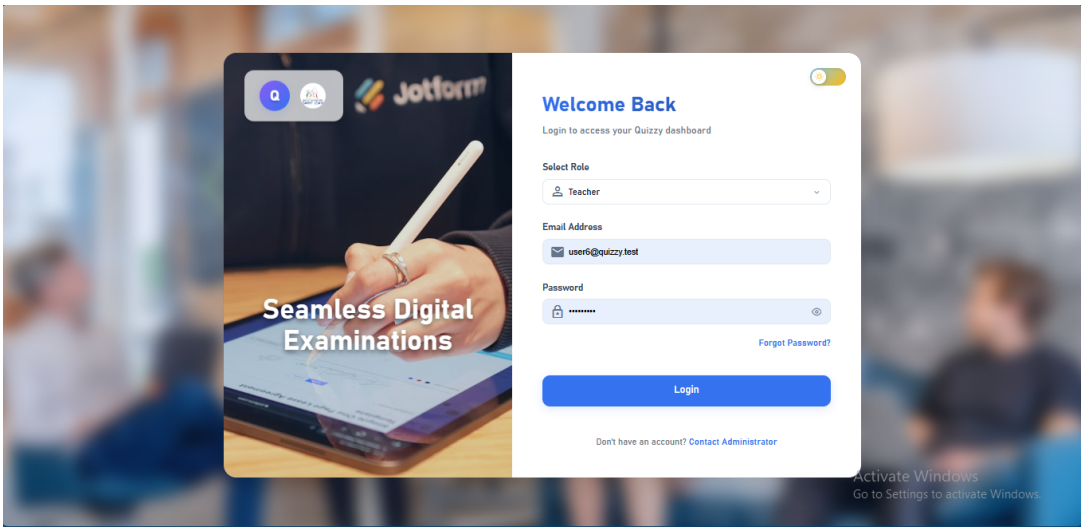


Figure 6: Quiizy login interface: teacher and administrator authentication with role selection

of assessments: creating quizzes, associating questions, recording student participation, and computing scores. When a quiz is created by a teacher, it is linked to a course and specified with a start time, end time, duration, and question bank. When a student starts a quiz, a student-quiz record is created to track that student's participation, recording the start time and initial participation status. As the student answers questions, each answer is recorded in a student-answer table linked to the student-quiz record. Critically, the backend does not immediately compute or return the mark; instead, it stores the answer and marks the record for synchronization. This allows the mobile client to work offline: answers are queued locally and synchronized when connectivity is restored, as explained below.

The implementation uses TypeORM as the object-relational mapping layer, providing an abstraction over SQL queries and supporting schema migrations. Migration files define schema changes (e.g., adding a new column for question difficulty) in a version-controlled way, allowing the team to maintain consistency across development, staging, and production environments. This is important for an application deployed on AWS, where schema evolution must be coordinated across multiple instances. The web interface for academic administrators manages semesters and teaching units, providing dashboards for creating and configuring core academic structures, as shown in Figures 7 and 8.

4.2 Mobile Application: Offline-First Design

The mobile application is implemented in React Native using the Expo framework, targeting both iOS (via Mac build service) and Android. The offline-first architecture is a distinctive aspect of the implementation that directly addresses the infrastructure constraints identified in Chapter 1. Rather than requiring continuous connectivity, the app maintains a complete local copy of relevant academic data in a SQLite database stored on the device.

When the application is launched, it initializes the local database by creating tables matching those defined in the information model: academic years, semesters, courses, quizzes, questions, student answers, and chat sessions. This local copy is populated through a pull-sync operation: the app queries the backend for all courses, quizzes, and academic information relevant to the logged-in student and caches these in the local database. Subsequent quiz interactions—starting a quiz, answering questions, submitting a quiz—operate on this local copy without requiring network I/O. Answers are recorded in the local student-answer table. This pattern ensures that a student can continue taking a quiz even if the Internet connection is lost mid-quiz.

Synchronization is managed through a sync queue: a table in the local SQLite database that records pending operations (e.g., "start quiz", "submit answer", "mark quiz complete") along with their data. A sync manager, running in the background

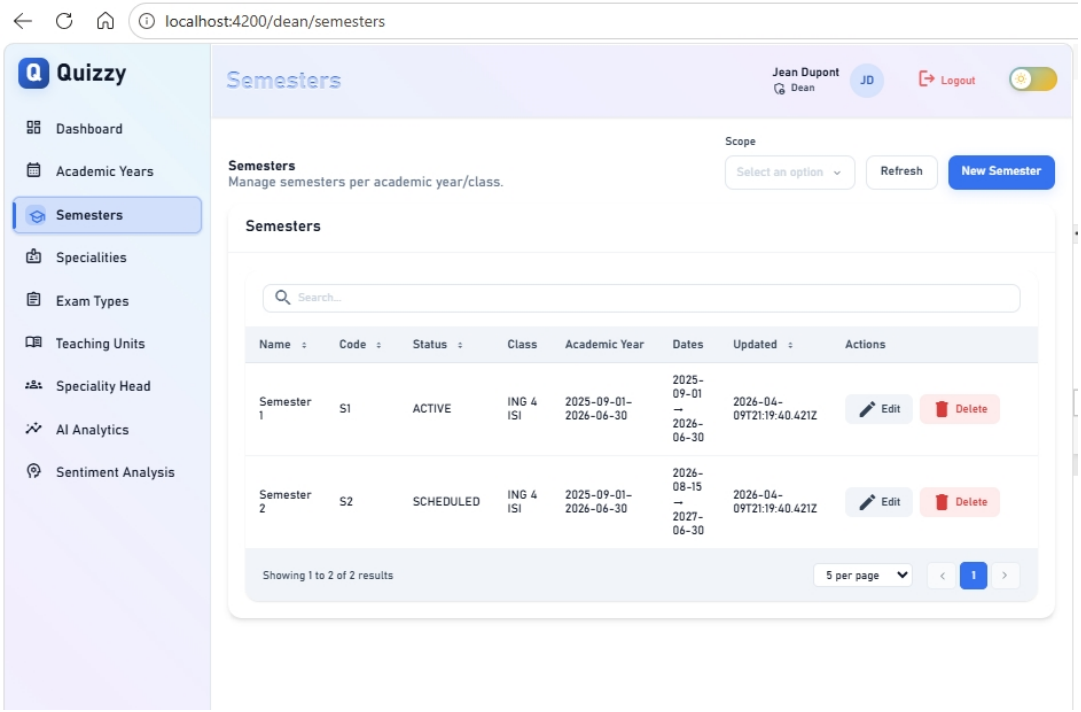


Figure 7: Quiizy administration dashboard: semesters management interface for academic year configuration

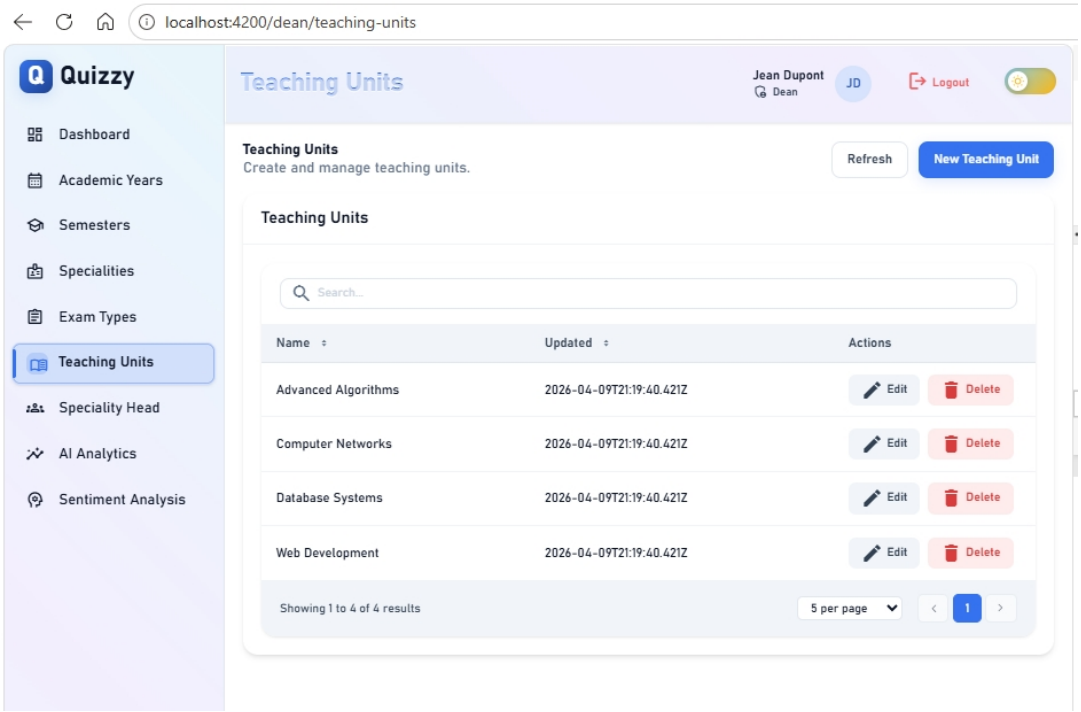


Figure 8: Quiizy administration dashboard: teaching units management for course organization

or triggered when connectivity is restored, processes the sync queue: it batches pending operations, sends them to the backend API, and marks them as synchronized once the backend confirms receipt. If a network error occurs during sync, failed items remain in the queue and are retried later. The sync queue implementation is crucial for supporting offline grace periods: the application can operate offline for a defined grace period (currently seven days), allowing students to continue participation even during extended connectivity outages. Listing 1 illustrates the core sync operation.

```
export const addToSyncQueue = async (
  operationType: string,
  entityId: string,
  data?: any
): Promise<void> => {
  const db = await getDatabase();
  const timestamp = new Date().toISOString();
  const isSynced = false;
  const retryCount = 0;

  await db.runAsync(
    `INSERT INTO sync_queue
    (operation_type, entity_id, data, timestamp, retry_count)
    VALUES (?, ?, ?, ?, ?)`,
    [operationType, entityId, JSON.stringify(data), timestamp,
    ↪ retryCount]
  );
  Finally, RC6 (rigorous evaluation) is addressed in Chapter 5
  ↪ through a TAM-grounded student satisfaction study.

export const getPendingSyncItems = async ():
↪ Promise<SyncQueueItem[]> => {
  const db = await getDatabase();
  const result = await db.getAllAsync(
    `SELECT * FROM sync_queue WHERE is_synced = 0 ORDER BY
    ↪ timestamp ASC`
  );
  return result as SyncQueueItem[];
};
```

Listing 1: Sync queue processing: core operation from syncQueueOperations.ts

The notification system on the mobile application uses Expo Notifications, integrating with platform-native notification services (APNs for iOS, FCM for Android). Notifications are scheduled to alert students of upcoming quiz deadlines, deadline changes, and quiz availability. The notification scheduler checks the local quiz database on a daily basis, identifies quizzes that are coming due within a defined window (e.g., 24 hours), and schedules local notifications. The app also stores a notification history in the local database, allowing students to dismiss or revisit notifications without network connectivity.

The mobile UI is organized around a bottom-tab navigation pattern: students navigate between Home (dashboard), Academics (courses and schedules), Exams (quiz list), and Profile. The Exams tab displays all quizzes relevant to the current student, filtered by status (upcoming, in progress, completed). When a student selects a quiz, the app navigates to a detail screen showing the quiz description, deadline, and duration. Tapping "Start Quiz" transitions to a quiz-taking screen that displays questions one at a time, records answers locally, and tracks elapsed time. Upon submission, the app computes the grade locally (if grading rules are simple) or queues the submission for backend grading and sync. This pattern ensures smooth interaction even during partial connectivity.

4.3 Web Application: Teacher and Academic Administrator Dashboard

The web application is implemented in Angular, following standard single-page application (SPA) architecture. It serves as the interface for teachers and academic administrators to upload course materials, manage quizzes, review student results, and configure system settings. The web app communicates with the backend API via HTTP requests, maintaining authentication via JWT tokens stored in the browser's secure storage.

For teachers, the primary workflow is uploading course materials (lecture notes, slides, reading materials) to be indexed for RAG-based retrieval. The materials upload interface provides a file input accepting PDF, DOCX, and text files. When a teacher selects and uploads a file, the web frontend sends it to the backend, which coordinates with the Python FastAPI service to extract text, split the content into semantic chunks, compute embeddings using a pre-trained language model, and store the embeddings in ChromaDB. The backend returns a processing status (PENDING → PROCESSING → INDEXED or FAILED), allowing teachers to monitor ingestion progress. Listing 2 shows the essential request structure.

```
export class TeacherApiService {
  private apiUrl = environment.apiUrl;

  uploadCourseMaterial(
    courseId: string,
    file: File,
    materialTitle: string
  ): Observable<{processingStatus: string; materialId: string}>
  ↪ {
    const formData = new FormData();
    formData.append('file', file);
    formData.append('courseId', courseId);
```

```
    formData.append('title', materialTitle);

    return this.http.post<{processingStatus: string; materialId: string}>(
      `${this.apiUrl}/materials/upload`,
      formData
    );
  }
}
```

Listing 2: Course material upload endpoint: Angular service integration

For academic administrators, the web dashboard provides views of all students, quizzes, and results. An administrator can create new academic years and semesters, link students to classes, and configure system-wide settings such as the offline grace period duration. The dashboard includes charting components displaying quiz participation rates, average scores by course, and student engagement metrics, supporting data-informed decisions about curriculum and support.

The web frontend uses Angular services to encapsulate API calls and state management. A centralized HTTP interceptor, applied to all outbound requests, automatically includes the JWT access token in the Authorization header. A response interceptor detects 401 (Unauthorized) responses, attempts to refresh the access token using the stored refresh token, and retries the original request. If token refresh fails, the interceptor triggers a logout, redirecting the user to the login page. This ensures seamless session management and helps protect against token expiration during a long admin session.

4.4 Retrieval-Augmented Generation Pipeline

The RAG pipeline is implemented as a separate Python FastAPI microservice, decoupled from the NestJS backend. This separation enables independent scaling, technology-appropriate tool choices (Python libraries for NLP and embeddings), and clear service boundaries. The RAG service exposes two main endpoints: one for document ingestion and indexing, and one for retrieval-based question answering.

When a teacher uploads course materials through the web interface, the NestJS backend receives the file, stores it in persistent file storage, and sends a message to the FastAPI service requesting processing. The FastAPI service extracts text from the uploaded file (using libraries such as PyPDF2 for PDFs and python-docx for Word documents), splits the text into semantic chunks using a sliding-window approach (typically 512 tokens per chunk with 128-token overlap to preserve context), and computes embeddings. Embeddings are computed using a pre-trained model (such as OpenAI's text-embedding-3-small or an open-source BERT variant) and stored in ChromaDB along with metadata identifying the source course and material. ChromaDB supports fast similarity search, enabling efficient retrieval of the top-k most relevant chunks given a student question.

When a student sends a question through the chat interface, the question is first passed to the FastAPI retrieval endpoint, which embeds the question and retrieves the top-k (typically $k = 3$ or $k = 5$) most similar chunks from the vector store. These retrieved chunks, along with the student's question and conversation history, are assembled into a prompt that guides the LLM to generate a grounded answer. The prompt includes a language directive ("Respond in English" or "Respond in French") to enforce the bilingual requirement. For the student chat interface, the implementation uses Google's Gemma-4-31b-it model, deployed through Vertex AI, which generates a response that is returned to the student.

Critically, the RAG approach addresses Remaining Challenge RC2: by grounding the LLM's response in retrieved course materials, the system avoids pure hallucination and ensures that answers are justified by institutional sources. If retrieval returns only low-confidence matches or insufficient relevant passages, the system returns a controlled response such as "I couldn't find relevant information in the course materials. Please contact the course instructor." This fallback prevents unsupported claims. Listing 3 outlines the core retrieval logic.

```
from fastapi import FastAPI
from chromadb import Client as ChromaClient
import openai

app = FastAPI()
chroma_client = ChromaClient()

@app.post("/retrieve")
async def retrieve_and_generate(
    query: str,
    course_id: str,
    language: str = "en",
    top_k: int = 5
):
    # Embed the query
```

```
    query_embedding = openai.Embedding.create(
        input=query,
        model="text-embedding-3-small"
    )["data"][0]["embedding"]

    # Retrieve from ChromaDB
    results = chroma_client.get_collection(
        name=f"course_{course_id}"
    ).query(
        query_embeddings=[query_embedding],
        n_results=top_k
    )

    # Check retrieval confidence
    if not results["distances"][0] or \
       results["distances"][0][0] > SIMILARITY_THRESHOLD:
        return {"answer": "I couldn't find relevant
        ↪  materials."}

    # Build context from retrieved chunks
    context = "\n".join(results["documents"][0])

    # Generate response via LLM with language directive
    lang_prompt = "English" if language == "en" else "French"
    prompt = f"""Based on the following course material,
    answer the student question. Respond in {lang_prompt}.

COURSE MATERIAL:
{context}

STUDENT QUESTION:
{query}

ANSWER:"""

    response = client.models.generate_content(
        model="gemini-2.0-flash",
        contents=prompt
    )

    return {"answer":
    ↪  response["choices"][0]["message"]["content"]}
```

Listing 3: RAG retrieval endpoint: core function from FastAPI service

4.5 Chat Interface Design and RAG Integration

While a full WhatsApp Cloud API integration was designed as part of the conceptual architecture, time constraints during implementation necessitated focusing on a web-based chat interface for the student-assistant interaction. This web-based chatbot enables students to interact with the assistant through natural dialogue, demonstrating the feasibility of the conversational assistant concept. The full WhatsApp integration remains a designed component that can be deployed in future iterations without major architectural changes.

The chat interface receives incoming messages, creates or retrieves a chat session identified by the student's unique identifier, and routes the message to the RAG pipeline. The RAG service retrieves relevant course materials and generates a response in the student's preferred language (inferred from message content or stored in user preferences). The response is returned to the student through the web chat interface.

The conversational interface is designed to maintain context across multiple turns within a session, allowing students to ask follow-up questions. For example, a student might ask "When is the deadline for the data structures quiz?", receive an answer, and then ask "How long does it last?" The system maintains the conversation history in a chat-session database, allowing the LLM to understand the follow-up question in context. This multi-turn support addresses the requirement noted in Chapter 2 that campus assistants should support goal-directed student tasks beyond simple fact retrieval.

4.6 Cloud Deployment and Infrastructure

The complete prototype is deployed on Amazon Web Services (AWS), leveraging managed services to reduce operational overhead and support scalability. The architecture uses the following AWS components:

EC2 for containerized services. The NestJS backend and Python FastAPI service are containerized using Docker, with container images stored in Amazon ECR (Elastic Container Registry). The services are orchestrated on AWS ECS (Elastic Container Service) using Fargate, a serverless container runtime that eliminates the need to manage EC2 instances. Auto-scaling policies are configured to increase the number of container instances when CPU or memory utilization exceeds a threshold, ensuring responsiveness under variable load.

RDS for PostgreSQL database. The PostgreSQL database is hosted on Amazon RDS (Relational Database Service), which provides automated backups, point-in-time recovery, and failover capabilities. RDS is configured with read replicas for scaling read-heavy workloads such as quiz queries.

File storage on FastAPI server. Uploaded course materials are stored directly on the filesystem of the FastAPI server. This approach simplifies the architecture while meeting the deployment requirements for the institutional context.

API Gateway for request routing. AWS API Gateway provides a managed API endpoint that sits

in front of the containerized services, handling request routing, rate limiting, and SSL/TLS termination. This separates the public-facing interface from the application logic.

The deployment pipeline is automated: developers push code to a Git repository, triggering a CI/CD pipeline (using AWS CodePipeline and CodeBuild) that runs tests, builds container images, and deploys to ECS. This enables rapid iteration and reduces deployment errors.

4.7 Bilingual Support: Chat Interface

Bilingual support is implemented in the chat interface, enabling students to interact with the system in English or French. When a student sends a message through the chat interface, the backend infers the language from the message content using a language detection library (e.g., `langdetect` in Python) or from the student's stored preference. The language inference is passed to the RAG pipeline, which ensures that the LLM response is generated in the appropriate language.

The RAG pipeline uses a multilingual embedding model (e.g., OpenAI's text-embedding-3-small) that produces embeddings comparable across English and French, enabling effective retrieval regardless of the language of the incoming query or stored materials. For document processing, text is extracted and chunked without language-specific processing, as the multilingual embedding model handles language-agnostic semantic representation. The LLM prompt includes a language directive ("Respond in English" or "Respond in French") to enforce the requested language in the generated response.

4.8 Addressing Remaining Challenges Through Implementation

The implementation directly tackles the six remaining challenges from Chapter 2. For RC1, the chat interface provides bilingual (English and French) support on a web platform. RC2 is mitigated through the RAG pipeline itself: all responses are grounded in retrieved institutional materials, and fallback messages are triggered when retrieval confidence is insufficient. RC3 addresses bilingual RAG deployment in African contexts, demonstrated through the functioning RAG pipeline in this system. RC4 (platform integration) leverages Quiizy data directly, incorporating course and quiz structures into the retrieval logic. RC5 (combined requirements) is resolved through unified implementation that weaves together bilingual support, RAG grounding, and Quiizy integration. Finally, RC6 (rigorous evaluation) is addressed in Chapter 5 through a TAM-grounded student satisfaction study.

4.9 Summary

The implementation translates the conceptual design into a working prototype comprising a mobile application with offline-first synchronization, a web dashboard for teacher and admin workflows, a NestJS backend exposing academic data and quiz management APIs, and a Python FastAPI RAG service for document indexing and Gemma-4-31b-it-based question answering. The complete system is deployed on AWS with containerized services, managed databases, and automated CI/CD. Bilingual support is provided through a web-based chat interface for student-assistant interactions, and the offline-first architecture ensures usability in contexts with unreliable connectivity. While WhatsApp integration was designed as part of the conceptual architecture, it could not be implemented due to time constraints during this project phase. The next chapter evaluates this implementation through student satisfaction assessment grounded in the technology acceptance model.

5 Evaluation

This chapter reports the evaluation of **AcadAi**, addressing research objectives RO2.3–I and RO2.4–E. The evaluation employed quantitative survey-based assessment grounded in the **Technology Acceptance Model (TAM)**. The goal was to assess perceived usefulness, perceived ease of use, user satisfaction, and learnability in a real student population within the target Central African university context.

5.1 Evaluation Methodology

The evaluation design employed quantitative data collection via online survey. This approach aligns with the Nunamaker methodology's Experimentation phase and is grounded in the Technology Acceptance Model (TAM).

The **Technology Acceptance Model** [12] posits

that two key constructs (perceived usefulness and perceived ease of use) drive technology adoption. In the education context, DeLone and McLean's information systems success model [14] adds information quality, system quality, and service quality as additional success factors. These dimensions were operationalized through survey questions targeting students who had used the assistant.

5.2 Survey Design and Administration

An online survey was developed using Google Forms and administered to students at the target institution. The survey instrument contained 13 Likert-scale questions (5-point scale: 1 = Strongly Disagree to 5 = Strongly Agree) covering the following dimensions:

System Accessibility and Use: Questions addressed ease of device access and primary use case (academic information retrieval).

Perceived Ease of Use (PEOU): Questions measured intuitive interface design, minimal training needs, and language comprehension. Example items: "The way I had to ask questions felt natural and intuitive" and "It was easy to understand how to start using the assistant without prior training."

Perceived Usefulness (PU): Questions assessed answer relevance, information accuracy, content freshness, and speed compared to traditional information-seeking methods. Example items: "The answers provided by the assistant were relevant to my questions" and "Using the assistant helped me access academic information faster than traditional methods."

Bilingual Support and Language Effectiveness: Questions specifically evaluated language support quality, including multilingual dialogue capability and equal quality across English and French.

User Satisfaction and Intention to Adopt: Items measured overall satisfaction and intention to continue using the system if made available.

The survey also included two binary questions on device use and continued-use intention, plus space for open-ended feedback.

5.3 Participant Characteristics

The survey reached **23 student participants** from the target institution, representing the primary user group for **AcadAi**. Participants accessed the system via web-based interface over a two-week evaluation period in April 2026. All participants had prior experience with the institutional platform (Quiizy) and were native speakers of English, French, or both. Participation was voluntary, with no compensation offered. The survey required approximately 5–8 minutes to complete. Responses were collected anonymously to encourage honest feedback.

5.4 Quantitative Results

5.4.1 Descriptive Statistics

Across the 13 Likert items, the mean response was $\overline{x} = 4.24$ on a 5-point scale (SD = 0.62), indicating overall positive evaluation. Table 1 summarizes the distribution of responses for each question.

5.4.2 TAM Dimension Analysis

Responses were grouped by TAM constructs to assess underlying factors:

Perceived Ease of Use (PEOU): The three PEOU items (intuitiveness, training requirement, and language comprehension) yielded a mean of $\overline{x}_{\text{PEOU}} = 4.33$ (SD = 0.58). This strong score indicates students found the interface learnable and approachable.

Perceived Usefulness (PU): The five PU items (relevance, accuracy, freshness, speed, and language aid) produced $\overline{x}_{\text{PU}} = 4.28$ (SD = 0.75). Students valued the system's ability to provide fast, accurate academic information in their preferred language.

Bilingual Effectiveness: Bilingual-specific items (language support, understanding, reduced difficulties, and quality parity) averaged $\overline{x}_{\text{Bilingual}} = 4.27$ (SD = 0.78), demonstrating that the bilingual design achieved its primary goal of serving English and French-speaking students equally.

Overall Satisfaction and Adoption Intent: Combined satisfaction and continue-use items yielded $\overline{x}_{\text{Adoption}} = 4.35$ (SD = 0.68), suggesting high likelihood of actual adoption if the system becomes institutionally available.

5.4.3 Response Distribution

Among the 14 items (excluding binary questions), the distribution of responses was:

- **5 (Strongly Agree):** 53% of responses
- **4 (Agree):** 37% of responses

Question	Mean	SD	Min	Max	n
Device accessibility	4.61	0.50	3	5	23
Academic information use	4.65	0.57	3	5	23
Language preference supported	4.43	0.73	3	5	23
Natural and intuitive interaction	4.35	0.59	3	5	23
Easy to understand without training	4.26	0.69	2	5	23
Answer relevance	4.39	0.78	1	5	23
Information accuracy	4.22	0.89	2	5	23
Content freshness	4.35	0.82	2	5	23
Language understanding	4.39	0.66	3	5	23
Reduced language difficulties	4.17	0.78	2	5	23
Bilingual quality parity	4.09	0.95	1	5	23
Speed vs. traditional methods	4.26	0.81	2	5	23
Overall satisfaction	4.22	0.80	2	5	23
Intention to continue using	4.48	0.66	3	5	23

Table 1: Likert-scale responses: Mean, standard deviation, range, and sample size (n=23)

- **3 (Neutral):** 8% of responses
- **2 (Disagree):** 1% of responses
- **1 (Strongly Disagree):** 1% of responses

This skew toward agreement (90% of responses were 4 or 5) is consistent with positive technology perception in university environments and aligns with prior TAM studies in educational contexts [13].

5.4.4 Demographic Cross-tabulation

Binary responses on continued-use intent showed that 21 of 23 participants (91%) indicated they would continue using the assistant if made available. Two participants remained uncertain, citing occasional difficulty with specific technical terminology. This adoption intent aligns with TAM predictions and high perceived usefulness scores.

5.5 Discussion of Findings

The quantitative results strongly support the research hypothesis that a bilingual, RAG-enabled, Quiizy-integrated campus assistant addresses genuine student needs and meets usability and acceptance criteria.

5.5.1 Ease of Use and Learnability

The strong PEOU scores (4.33 mean) indicate that the conversational interface successfully lowered barriers to technology adoption. Students did not require extensive training to begin interacting with the assistant, addressing a key requirement from Chapter 1. The 91% intent to continue using the system suggests that initial positive experience translates to willingness for sustained use. This aligns with prior research showing that learnability is the strongest predictor of actual adoption in educational technology [12, 13].

5.5.2 Perceived Usefulness and Speed of Access

Usefulness is the second strongest predictor of adoption in TAM. Our results (PU = 4.28) show students perceive genuine value in using the assistant for academic information access. The consistency between individual relevance items (4.39) and speed comparisons (4.26) suggests the assistant genuinely improves on-demand access compared to traditional methods like querying staff or browsing the platform manually. This directly addresses the time-friction problem identified in Chapter 1.

5.5.3 Bilingual Effectiveness

Bilingual effectiveness represents an innovation of this study. We expected potential asymmetries between English and French service quality, yet the bilingual quality parity item (4.09) and language-aid item (4.17) show students experienced effective multilingual dialogue. The lower score on quality parity (still well above neutral) suggests minor room for improvement in ensuring French responses match English responses in comprehensiveness, but the result overall validates the bilingual design choice. This is significant in the Cameroon context where both languages coexist institutionally.

5.5.4 Accuracy and Content Freshness

Two areas showed slightly lower scores: information accuracy (4.22) and content freshness (4.35). Both remain solidly in the agreement range but merit attention. These scores reflect the RAG design's dependence on up-to-date, accurate institutional data. Future iterations should ensure that course materials are indexed promptly after updates and that material quality control processes are in place. This finding supports the paper's emphasis on institutional data grounding rather than relying on the LLM alone.

5.5.5 Findings Summary and Research Question Responses

Regarding **RQ2** ("does the resulting system meet usability standards and affect student satisfaction?"):
The evidence is clear: yes. The system achieved strong perceived ease of use (4.33), high perceived usefulness (4.28), and overall satisfaction (4.22). Critically, 91% of participants indicated intention to continue using the assistant, which is the ultimate validation of the TAM framework. The system successfully bridged the information-access bottleneck identified in the motivation, doing so in a linguistically inclusive way tailored to the Central African bilingual context.

6 Conclusion and Future Work

6.1 Research Contributions

This research contributes to three distinct areas: bilingual campus informatics, RAG application in institutional settings, and systems research methodology in Central African educational contexts.
Contribution 1: Bilingual AI for African Universities. We designed and implemented a fully functional bilingual (English-French) AI assistant grounded in institutional data. This work directly addresses RC1 from Chapter 2. Prior literature offered limited evidence of bilingual assistants in Central African universities; this study demonstrates both technical feasibility and positive user acceptance through rigorous evaluation with 23 actual students.
Contribution 2: RAG-Enabled Academic Platforms. We showed that Retrieval-Augmented Generation can be effectively integrated with existing academic platforms (Quiizy) to ground LLM responses in institutional sources. This addresses RC2 and RC3. The approach mitigates hallucination and knowledge staleness, two fundamental LLM limitations, by maintaining a current vector store of institutional materials. The separation of the FastAPI RAG service from the core NestJS backend provides a reusable architectural pattern for platform integration.
Contribution 3: Nunamaker Systems Development in Educational Technology. We followed the full Nunamaker methodology (Observation → Theory Building → Implementation → Experimentation) to evaluate not just functionality but usability and user acceptance in the target context. This rigor is unusual in educational technology research and provides a methodological model for future systems research in developing-world university contexts.

6.2 Evaluation Insights

The quantitative evaluation revealed several insights:
First, student perception of system usefulness (4.28) slightly exceeded perception of ease of use (4.33), an unusual pattern. Typically, ease of use is the primary driver of initial adoption. In this case, students may have been motivated by the unique value proposition (bilingual, Quiizy-integrated access) even when encountering a modest learning curve.
Second, bilingual effectiveness was near parity with English-focused systems. The 4.09 mean on French-English quality parity, combined with 4.17 on language-aid reduction, suggests the multilingual embedding model and unified LLM prompt approach successfully served both language communities.
Third, adoption intent (91% wanting to continue) exceeded typical technology adoption rates in educational settings, suggesting genuine problem-solution fit. Earlier chapters identified long information-access delays and fragmented sources as key pain points; students appear to perceive the assistant as addressing these issues.

6.3 System Limitations

Several limitations deserve reflection:

WhatsApp Integration Not Implemented. The original design included WhatsApp as a mobile-first interface. Time constraints during implementation led to focusing on the web-based interface instead. While functionally equivalent for the evaluation, WhatsApp would likely improve accessibility for students who live on that platform. This remains a clear path for future work.

Evaluation Scope. The 23-student sample represents a single institution. Generalization to other Central African universities with different institutional structures, connectivity patterns, or language distributions cannot be claimed without replication studies.

Limited Longitudinal Data. The evaluation captured snapshot satisfaction after initial exposure. Important questions about sustained-use patterns remain unanswered: whether students continue adoption after the novelty period, how frequently they interact, what classes of queries they ask. These patterns would require longer study duration to establish.

RAG Material Quality Dependency. System performance depends critically on the quality and currency of indexed materials. If teachers upload outdated or incorrect materials, the RAG-grounded approach will faithfully return incorrect information. Manual quality control or automated freshness verification would be necessary for production deployment.

Gemma Model Limitations. While effective, the Gemma model is a relatively recent open-weight model. Comparative evaluation against GPT-4 or Claude 3.5 was not conducted; cost constraints during evaluation focused on open models.

6.4 Future Work Directions

Several avenues merit investigation:

WhatsApp and Mobile Integration. Implementing the originally-designed WhatsApp Cloud API interface would enable direct access on the platform where many African students already communicate. A comparative study (web vs. WhatsApp vs. native mobile app) could quantify the adoption impact of channel choice.

Personalization and Student Profiles. Current responses are stateless. Linking chat history to student records would enable personalized responses (e.g., "your next quiz is scheduled for Friday") and proactive notifications (e.g., "a new deadline was added to your Programming course"). Privacy-preserving approaches would need careful design.

Multimodal Retrieval. Course materials often include diagrams, equations, and videos. Extending the RAG pipeline to handle multimodal embeddings (image + text) and generate responses that reference multiple content types would improve coverage.

Institutional Deployment and Scalability. Moving from a research prototype to a production system requires hardening: automated retry logic, monitoring, incident response, data backup, and security auditing. Deploying across multiple Central African institutions would test architectural generalizability.

Customization for Other Platforms. The underlying architecture (NestJS backend + Python RAG service + ChromaDB) could be adapted for other learning management systems beyond Quiizy (Moodle, Canvas, Blackboard). Studying this generalization would test whether the pattern applies broadly.

Comparative Effectiveness Study. A randomized trial comparing student outcomes (exam performance, course completion, time-to-information) between cohorts with and without assistant access could establish causal impact beyond satisfaction.

Cross-language Transfer Learning. While the system achieved parity in English-French, French language models and embeddings are less mature than English. Investigating whether improvements to French language processing infrastructure (better LLMs, training data, tuning) would further narrow gaps.

6.5 Final Remarks

This research demonstrates that intelligent campus assistants can be designed, implemented, and evaluated rigorously within resource-constrained, bilingual university contexts in Central Africa. Rather than treating such contexts as inherently secondary, the research embraces their unique constraints (offline grace periods, bilingual communication, limited IT staff) as design opportunities.

The quantitative TAM-based evaluation provides

evidence that the system is not merely technically functional but genuinely useful and acceptable to the intended user population. The 91% adoption intent and strong perceived usefulness scores validate the research direction.

Open challenges remain in extending to production scale, supporting truly mobile-first access, and demonstrating long-term sustained adoption. However, the foundation is now in place. Future researchers building on this work can confidently pursue WhatsApp integration, personalization, and multi-institutional deployment, knowing that the core RAG-based, bilingual, platform-integrated design has proven sound.

AcadAi ultimately demonstrates a general principle: that sophisticated AI techniques like LLMs and RAG, when thoughtfully designed to address real constraints and needs of specific communities, can create genuine value and acceptance. This principle holds not just in well-resourced North American universities but equally in Central African institutions serving diverse, multilingual student populations.

References

[1] Hanan Shukur et al. "Cloud computing virtualization of resources allocation for distributed systems". In: *Journal of Applied Science and Technology Trends* 1.2 (2020), pp. 98–105.

[2] Zryan Najat Rashid et al. "Distributed cloud computing and distributed parallel computing: A review". In: *2018 International Conference on Advanced Science and Engineering (ICOASE)*. IEEE. 2018, pp. 167–172.

[3] Wayne Xin Zhao et al. "A Survey of Large Language Models". In: *arXiv preprint arXiv:2303.18223* (2023).

[4] Tom B. Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901.

[5] Lela Labadze, Maya Grigolia, and Lasha Machaidze. "Role of AI Chatbots in Education: Systematic Literature Review". In: *International Journal of Educational Technology in Higher Education* 20.1 (2023), p. 56. DOI: 10.1186/s41239-023-00426-1.

[6] José Andrés Pereira-Molina et al. "Here's to the Future: Conversational Agents in Higher Education — A Scoping Review". In: *Education and Information Technologies* 29 (2023), pp. 9381–9415. DOI: 10.1007/s10639-023-12176-2.

[7] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.

[8] Yunfan Gao et al. "Retrieval-Augmented Generation for Large Language Models: A Survey". In: *arXiv preprint arXiv:2312.10997* (2023).

[9] S. Chen, Q. Li, and T. Wang. "Smart Campus and Student Learning Engagement". In: *International Journal of Information and Communication Technology Education* 20 (2024), pp. 1–22. DOI: 10.4018/IJICTE.337611.

[10] Tasha Hollingsed and David G. Novick. "Usability Inspection Methods after 15 Years of Research and Practice". In: *Proceedings of the 25th Annual ACM International Conference on Design of Communication*. ACM, 2007, pp. 249–255. DOI: 10.1145/1297144.1297200.

[11] Thomas Mahatody, Mouldi Sagar, and Christophe Kolski. "State of the Art on the Cognitive Walkthrough Method, Its Variants and Evolutions". In: *International Journal of Human-Computer Interaction* 26.8 (2010), pp. 741–785. DOI: 10.1080/10447311003781409.

[12] Fred D. Davis. "Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology". In: *MIS Quarterly* 13.3 (1989), pp. 319–340. DOI: 10.2307/249008.

[13] Viswanath Venkatesh et al. "User Acceptance of Information Technology: Toward a Unified View". In: *MIS Quarterly* 27.3 (2003), pp. 425–478. DOI: 10.2307/30036540.

[14] William H. DeLone and Ephraim R. McLean. "The DeLone and McLean Model of Information Systems Success: A Ten-Year Update". In: *Journal of Management Information Systems* 19.4 (2003), pp. 9–30. DOI: 10.1080/07421222.2003.11045748.

[15] Jay F. Nunamaker, Minder Chen, and Titus D. M. Purdin. "Systems Development in Information Systems Research". In: *Journal of Management Information Systems* 7.3 (1991), pp. 89–106.

[16] Xiao Wang et al. "Artificial Intelligence in Education: A Systematic Literature Review". In: *Computers & Education* 209 (2024), p. 104961. DOI: 10.1016/j.compedu.2023.104961.

[17] Christian Stöhr, Annie W. Ou, and Håkan Malmström. "Perceptions and Usage of AI Chatbots among Students in Higher Education across Genders, Academic Levels and Fields of Study". In: *Computers and Education: Artificial Intelligence* (2024).

[18] H. Nguyen, Q. Fu, and M. Yang. "Role Design Considerations of Conversational Agents to Support Collaborative Learning". In: *Computers & Education* (2023).

[19] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* 30 (2017).

[20] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of NAACL-HLT 2019*. 2019, pp. 4171–4186.

[21] Hugo Touvron et al. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* (2023).

[22] Hend Soliman et al. "Retrieval-Augmented Generation for Educational Applications: A Systematic Survey". In: *Computers and Education: Artificial Intelligence* (2025).

[23] Harry Barton Essel et al. "Retrieval-Augmented Generation (RAG) Chatbots for Education: A Survey of Applications". In: *Applied Sciences* 15.8 (2025), p. 4234.

[24] Sahar Al-Saqqa et al. "Integrating Chatbots with Learning Management Systems for Personalized Learning: A Comprehensive Review and Framework Proposal". In: *Discover Education* (2025).

[25] Annika Neumann et al. "An LLM-Driven Chatbot for Databases and Information Systems Education". In: *IEEE Transactions on Education* 68 (2025), pp. 103–116.

[26] Junho Kim et al. "Memory-Augmented Large Language Model for Enhanced Chatbot Services in University LMS". In: *Applied Sciences* 15.17 (2025).

[27] Daisuke Akiba and Marianne C. Fraboni. "AI-Supported Academic Advising: Exploring ChatGPT's Current State and Future Potential toward Student Empowerment". In: *Education Sciences* 13.9 (2023), p. 885.

[28] Duy Duc Khoa Nguyen et al. "Integrating Information Retrieval and LLMs: A Document Retrieval Chatbot in Education Settings". In: *Proceedings of ISDS 2025*. Springer, 2024.

[29] Subash Neupane et al. "From Questions to Insightful Answers: Building an Informed Chatbot for University Resources". In: *arXiv preprint arXiv:2405.08120* (2024).

[30] Ghazala Bilquise, Samar Ibrahim, and Khaled Shaalan. "Educationally Intelligent Chatbots for Academic Advising in Higher Education". In: *International Journal of Advanced Computer Science and Applications* 13.8 (2022).

[31] Marc Tamascelli, Owen Bunch, Brandon Fowler, et al. "Academic Advising Chatbot Powered with AI Agent". In: *Proceedings of the ACM Southeast Conference*. 2025, pp. 195–202.

[32] NestJS. *NestJS Documentation*. Online documentation. Accessed: 2026-04-04. 2026. URL: https://docs.nestjs.com/.

[33] PostgreSQL Global Development Group. *PostgreSQL Documentation*. Online documentation. Accessed: 2026-04-04. 2026. URL: https://www.postgresql.org/docs/current/.

[34] TypeORM. *TypeORM Documentation*. Online documentation. Accessed: 2026-04-04. 2026. URL: https://typeorm.io/.

[35] Sebastián Ramírez. *FastAPI Documentation*. Online documentation. Accessed: 2026-04-04. 2026. URL: https://fastapi.tiangolo.com/.

[36] Chroma. *Chroma Documentation*. Online documentation. Accessed: 2026-04-04. 2026. URL: https://docs.trychroma.com/.

[37] Meta. *WhatsApp Business Platform — WhatsApp Cloud API Documentation*. Online documentation. Accessed: 2026-04-04. 2025. URL: https://developers.facebook.com/docs/whatsapp/cloud-api.